

SHILATE

Software Hardware In Loop Automotive Testing Environment

Tudor-Ciprian Tămaş

Supervisor: Sl. Dr. Ing. Valentin Mărănescu

Electronics, Telecommunications and Information Technology
Politehnica University of Timișoara

July 2026



Motivation

The Industry Shift

- Vehicles are becoming **software-defined** — behaviour updated over-the-air
- Reinforcement learning (RL) is maturing as a practical control tool
- But no open-source pipeline bridges **RL training** and **SDV deployment**

Four Open Challenges

- 1 Simulation $\leftrightarrow$ middleware protocol coupling
- 2 Standardised signal representation (VSS)
- 3 Operational safety monitoring of learned policies

Goal

Build an **end-to-end, open-source** pipeline that:

- Trains a deep RL agent in a Unity physics sim
- Deploys the policy onto a real Eclipse Leda SDV stack
- Uses the **same MQTT / VSS layer** from training to hardware

System Architecture — Three Loosely Coupled Tiers

Tier 1: Unity Simulation

Vehicle physics · LiDAR
· Training Controller



Mosquitto MQTT broker



Tier 2: Python PPO

- Stable-Baselines3
 - Gymnasium env

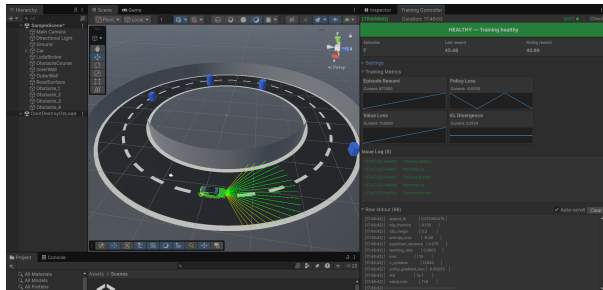


Mosquitto MQTT broker



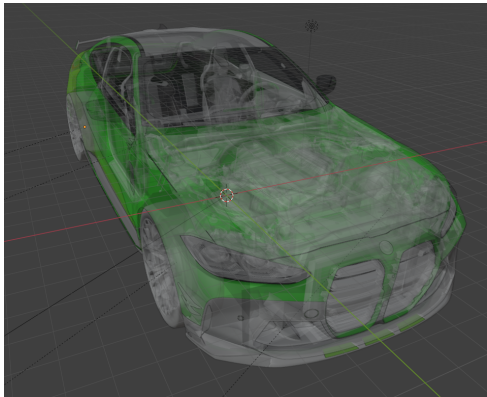
Tier 3: Eclipse Leda SDV

Kuksa · feeder ·
Velocitas · RPi 4



Simulation environment overview

- All inter-tier traffic flows through **MQTT topic prefix env0/**
- No direct function calls across tiers — fully decoupled
- Same topic namespace in training and on



Rear-wheel-drive vehicle model

Vehicle & Sensor

- 1 500 kg RWD chassis — four WheelColliders, Pacejka tire model
- **21-ray LiDAR** spanning 120° at 50 m range, published at 10 Hz
- Observation $o \in \mathbb{R}^{23}$: rays + normalised speed + steer

Reward Function

$$r_t = \underbrace{2 \Delta p_t}_{\text{progress}} + \underbrace{(-5) 1_{\text{collision}}}_{\text{collision}} + \underbrace{+50 1_{\text{lap}} + +25 1_{\frac{1}{2}\text{-turn}}}_{\text{milestones}}$$

Training Controller (Editor window)

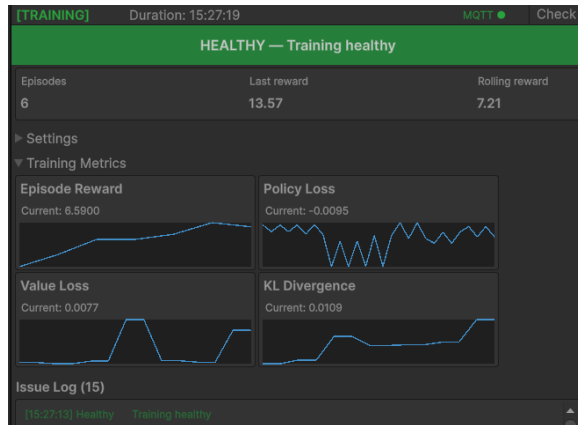
Python RL Agent — PPO via Stable-Baselines3

Algorithm & Architecture

- Proximal Policy Optimisation (PPO)
- Actor-Critic MLP: two hidden layers [256, 128]
- Action space: $a = [\delta, \theta, \beta]$ — steer $\in [-1, 1]$, throttle $\in [0, 1]$, brake $\in [0, 1]$

Key Hyperparameters

Parameter	Value
Total timesteps	100 000
Learning rate	3×10^{-4}
Rollout steps	2 048
Batch size	64
Clip ϵ	0.2



Training convergence graphs (TensorBoard)

- Gymnasium-compliant wrapper — any SB3 algorithm can be substituted

Eclipse Leda SDV Integration

```
[ OK ] Started Mosquitto MQTT Broker.
[ OK ] Started Eclipse Kanto - Update Manager.
[ OK ] Started containerd container runtime.
[ OK ] Started Eclipse Kanto - Container Management.

Eclipse Leda v0.1.0-M3
leda-525400123456 login: root

[LED]
Eclipse Leda v0.1.0-M3 (GNU/Linux 5.15.124-yocto-standard x86_64)

Hostname.....: leda-525400123456
Uptime.....: 0 days, 0:0 hours
System load....: 0.63 (1min) | 0.15 (5min) | 0.05 (15min)
Disk Space....: Root: 698.7M | Used: 518.8M | Free: 128.1M
                  Data: 8.6G | Used: 4.8G | Free: 3.5G
RAM .....: Total: 3.8G | Used: 120.8M | Free: 3.6G
Network.....: not connected or activated!
To check SDV services health, run $ sdv-health
Use $ sdv-device-info to display device configuration
Use $ sdv-provision to generate self-signed device certificates
```

Eclipse Leda container stack

5-Container Docker Compose Stack

- 1 **Mosquitto** — MQTT broker
- 2 **Kuksa databroker** — VSS signal store

Signal Chain

- env0/vehicle/speed mapped to Vehicle.Speed (COVESA VSS)
- Kuksa databroker stores all VSS values
- Velocitas app enforces safety limits:
 - Overspeed alert if speed > 120 km/h
 - RPM alert if RPM > 6 500

Deployment to Raspberry Pi 4

- deploy-pi.sh — ARM64 cross-compile via docker buildx
- Transfers images over SSH/SCP, starts stack with docker compose up -d
- Full deploy cycle: ≈ 14 minutes

Experimental Results

Training Convergence (3 seeds, mean)

Milestone	Timestep	σ
First collision-free episode	12 400	$\pm 2\,100$
Half-turn bonus triggered	31 000	$\pm 3\,800$
First full lap	48 500	$\pm 5\,200$
Rolling mean ≥ 60	76 000	$\pm 6\,900$

Inference — 50 episodes (local)

Metric	Value
Lap completion rate	82%
Mean episode reward	63.4
Mean lap duration	38.2 s
Mean lap time (local)	17.6 s (4)

Ablation Study

- Half-turn bonus removed ($c_h = 0$): first lap delayed by $\approx 18\,000$ steps; rolling mean 12 pts lower at 100K
- Weaker collision penalty ($c_c = -1$): laps completed but agent drove closer to walls

Health Monitoring — All Failure Modes Detected

- Observation timeout (> 5 s) $\rightarrow$ *Critical*
- Python crash $\rightarrow$ non-zero exit code
- MQTT disconnect $\rightarrow$ *Disconnected* within 35 s

Raspberry Pi Remote Inference

Conclusions

What was achieved

- End-to-end RL → SDV pipeline — same MQTT/VSS protocol from Unity training to Raspberry Pi 4 hardware
- Agent converges within 100 000 timesteps; 82% lap completion in inference
- One-click Training Controller with 5-failure-mode health monitoring
- Validated on physical hardware — 74% remote lap completion over LAN
- Curriculum scaling: automatic obstacle density increase with agent skill
- Parallel training via Headless Orchestrator (up to 30

Future Work

- Domain randomisation for sim-to-real transfer
- Safety monitor → active command override (not just alerts)
- Multi-agent scenarios and cooperative driving
- CAN bus hardware-in-loop testing on real ECUs